

```

In [36]: import os
import time
from pathlib import Path

import numpy as np
import pandas as pd
import requests

os.environ["CENSUS_API_KEY"] = "b1278b466e026ad679a62a03dd4b044bb8b36e7c"
CENSUS_API_KEY = os.getenv("CENSUS_API_KEY")
print("Key loaded:", CENSUS_API_KEY is not None)

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)
pd.set_option("display.max_colwidth", 120)

# -----
# Paths / config
# -----
DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUT_DIR = DATA_DIR / "processed"
RAW_DIR.mkdir(parents=True, exist_ok=True)
OUT_DIR.mkdir(parents=True, exist_ok=True)

# Update this path if needed
IGS_PATH = r"C:\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_cha

# Tract selection for focused EDA / testing
TARGET_TRACTS = None
# Example:
# TARGET_TRACTS = ["13089023301", "13089023405", "13089023302"]

# If TARGET_TRACTS is set, the code will infer state FIPS from those tracts
SAVE_INTERMEDIATE = True

# ACS 5-year currently available through 2024
ACS_MAX_AVAILABLE_YEAR = 2024
FILL_2025_WITH_2024_ACS = False # optional, keep False by default

```

Key loaded: True

Cell 2 — IGS load / clean helpers

```

In [37]: IGS_STRING_COLS = {
    "County",
    "State",
    "BENCHMARK",
    "Census Tract FIPS code",
    "geoid",
}

IGS_ALIAS_MAP = {
    "Inclusive Growth Score": "igs_total",

```

```

    "Growth": "igs_growth",
    "Inclusion": "igs_inclusion",
    "Place": "igs_place",
    "Economy": "igs_economy",
    "Community": "igs_community",
    "Place Growth": "igs_place_growth",
    "Place Inclusion": "igs_place_inclusion",
    "Economy Growth": "igs_economy_growth",
    "Economy Inclusion": "igs_economy_inclusion",
    "Community Growth": "igs_community_growth",
    "Community Inclusion": "igs_community_inclusion",
}

def dedupe_columns(cols):
    seen = {}
    out = []
    for c in cols:
        c = str(c).strip()
        if c not in seen:
            seen[c] = 0
            out.append(c)
        else:
            seen[c] += 1
            out.append(f"{c}.{seen[c]}")
    return out

def _find_header_row(df_raw, required_tokens=("Census Tract FIPS", "Year")):
    tokens = [t.lower() for t in required_tokens]
    for i in range(min(len(df_raw), 50)):
        row = df_raw.iloc[i].astype(str).str.lower().tolist()
        if all(any(tok in cell for cell in row) for tok in tokens):
            return i
    raise ValueError("Could not find the real IGS header row.")

def normalize_geoid_series(series: pd.Series) -> pd.Series:
    s = series.astype("string").str.strip()

    numeric = pd.to_numeric(s, errors="coerce")
    numeric_mask = numeric.notna()

    s = s.copy()
    s.loc[numeric_mask] = numeric.loc[numeric_mask].astype("Int64").astype("string")

    s = s.str.replace(r"\.0$", "", regex=True)
    s = s.str.replace(r"\D", "", regex=True)
    s = s.str.zfill(11)
    s = s.where(s.str.fullmatch(r"\d{11}"), pd.NA)

    return s

def filter_tracts(df: pd.DataFrame, tracts=None, geoid_col="geoid") -> pd.DataFrame:
    if not tracts:

```

```

        return df.copy()

    tract_set = {str(t).zfill(11) for t in tracts}
    out = df[df[geoid_col].astype("string").isin(tract_set)].copy()
    return out

def safe_save_parquet(df: pd.DataFrame, path: Path):
    out = df.copy()

    for c in out.columns:
        if out[c].dtype == "object":
            out[c] = out[c].astype("string")

    out.to_parquet(path, index=False, engine="pyarrow")

def load_igs_any(path: str) -> pd.DataFrame:
    p = Path(path)

    if p.suffix.lower() in [".xlsx", ".xls"]:
        raw = pd.read_excel(p, header=None)
    else:
        raw = pd.read_csv(p, header=None, low_memory=False)

    hdr_i = _find_header_row(raw, required_tokens=("Census Tract FIPS", "Year"))
    header = dedupe_columns(raw.iloc[hdr_i].tolist())

    df = raw.iloc[hdr_i + 1 :].copy()
    df.columns = header
    df = df.dropna(how="all").reset_index(drop=True)

    # Drop any duplicated header rows that still appear in the body
    if "Census Tract FIPS code" in df.columns:
        df = df[
            df["Census Tract FIPS code"].astype(str).str.strip().str.lower()
            .copy()

    # Normalize GEOID + year
    df["geoid"] = normalize_geoid_series(df["Census Tract FIPS code"])
    df["year"] = pd.to_numeric(df["Year"], errors="coerce")

    # Convert non-ID columns to numeric where possible
    for c in df.columns:
        if c not in IGS_STRING_COLS:
            df[c] = pd.to_numeric(df[c], errors="coerce")

    # Friendly aliases for easier modeling later
    for old_col, new_col in IGS_ALIAS_MAP.items():
        if old_col in df.columns:
            df[new_col] = pd.to_numeric(df[old_col], errors="coerce")

    if "Is an Opportunity Zone" in df.columns:
        df["is_opp_zone"] = pd.to_numeric(df["Is an Opportunity Zone"], errors="coerce")

    if "URBAN CODE" in df.columns:

```

```

df["urban_code"] = pd.to_numeric(df["URBAN CODE"], errors="coerce")

if "BENCHMARK" in df.columns:
    df["benchmark"] = df["BENCHMARK"].astype("string")

# Final cleanup
df = df.dropna(subset=["geoid", "year"]).copy()
df["year"] = df["year"].astype(int)
df = df.sort_values(["geoid", "year"]).reset_index(drop=True)

return df

```

Cell 3 — Load and save clean IGS

```

In [38]: igs = load_igs_any(IGS_PATH)
igs = filter_tracts(igs, TARGET_TRACTS)

if SAVE_INTERMEDIATE:
    safe_save_parquet(igs, OUT_DIR / "igs_clean.parquet")

print("IGS shape:", igs.shape)
print("IGS years:", sorted(igs["year"].dropna().unique().tolist())[:10], "...")
print()
print(igs[["geoid", "year", "igs_total"]].head())
print()
print(igs[["igs_total", "igs_place", "igs_economy", "igs_community"]].describe())
print()
print("Top missingness:")
print(igs.isna().mean().sort_values(ascending=False).head(20))

```

```
IGS shape: (765288, 87)
IGS years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025] ... [2023, 2024, 2025]
```

```

      geoid  year  igs_total
0  01001020100  2017      47.0
1  01001020100  2018      52.0
2  01001020100  2019      46.0
3  01001020100  2020      46.0
4  01001020100  2021      38.0

```

	igs_total	igs_place	igs_economy	igs_community
count	757582.000000	761912.000000	764294.000000	762407.000000
mean	50.052242	49.848921	50.328149	50.148915
std	10.252555	12.453479	13.532944	13.744805
min	0.000000	0.000000	0.000000	0.000000
25%	44.000000	42.000000	42.000000	40.000000
50%	50.000000	50.000000	51.500000	50.000000
75%	57.900000	58.000000	60.000000	60.000000
max	84.000000	100.000000	94.000000	100.000000

```

Top missingness:
Is an Opportunity Zone                1.000000
Spend Growth Base, %                  1.000000
is_opp_zone                           1.000000
Spending per Capita Base, %           1.000000
Spending per Capita Tract, %          1.000000
Spend Growth Tract, %                 1.000000
Early Education Enrollment Tract, %    0.176266
Early Education Enrollment Score       0.176266
Minority/Women Owned Businesses Base, % 0.159723
Minority/Women Owned Businesses Score  0.159723
Minority/Women Owned Businesses Tract, % 0.159723
Spend Growth Score                     0.099624
Spending per Capita Score              0.099624
Residential Real Estate Value Score    0.050105
Residential Real Estate Value Tract, % 0.050105
Travel Time to Work Tract, %          0.024643
Travel Time to Work Score             0.024643
Affordable Housing Score               0.019497
Affordable Housing Tract, %            0.019497
Female Above Poverty Tract, %          0.018882
dtype: float64

```

Cell 4 — ACS variables to pull

```

In [39]: ACS_VAR_GROUPS = {
    "population_age": [
        "B01001_001E", # total pop
        "B01001_003E", "B01001_004E", "B01001_005E", "B01001_006E", # male
        "B01001_027E", "B01001_028E", "B01001_029E", "B01001_030E", # female
        "B01001_020E", "B01001_021E", "B01001_022E", "B01001_023E", "B01001_
        "B01001_044E", "B01001_045E", "B01001_046E", "B01001_047E", "B01001_
    ],
    "race": [

```

```

        "B02001_001E",
        "B02001_002E",
        "B02001_003E",
        "B02001_005E",
        "B02001_008E",
    ],
    "education": [
        "B15003_001E",
        "B15003_022E", "B15003_023E", "B15003_024E", "B15003_025E",
    ],
    "employment": [
        "B23025_001E",
        "B23025_002E",
        "B23025_003E",
        "B23025_005E",
    ],
    "income_poverty": [
        "B19013_001E", # median household income
        "B19301_001E", # per-capita income
        "B19083_001E", # gini
        "B17001_001E", # poverty universe
        "B17001_002E", # below poverty
    ],
    "commute": [
        "B08303_001E",
        "B08303_002E", "B08303_003E", "B08303_004E", "B08303_005E",
        "B08303_006E", "B08303_007E", "B08303_008E",
    ],
    "internet": [
        "B28002_001E",
        "B28002_002E",
    ],
    "housing": [
        "B25002_001E", "B25002_002E", "B25002_003E", # total / occupied /
        "B25003_001E", "B25003_002E", "B25003_003E", # tenure
        "B25064_001E", # median gross rent
        "B25077_001E", # median home value
    ],
    "rent_burden": [
        "B25070_001E",
        "B25070_002E", "B25070_003E", "B25070_004E", "B25070_005E", "B25070_
    ],
    "owner_cost_burden": [
        "B25091_001E",
        "B25091_003E", "B25091_004E", "B25091_005E", "B25091_006E", "B25091_
        "B25091_014E", "B25091_015E", "B25091_016E", "B25091_017E", "B25091_
    ],
    "early_education": [
        "B14003_004E", "B14003_013E", "B14003_032E", "B14003_041E",
    ],
    "health_insurance": [
        "B27001_001E",
        "B27001_005E", "B27001_008E", "B27001_011E", "B27001_014E", "B27001_
        "B27001_020E", "B27001_023E", "B27001_026E", "B27001_029E",
        "B27001_033E", "B27001_036E", "B27001_039E", "B27001_042E", "B27001_
        "B27001_048E", "B27001_051E", "B27001_054E", "B27001_057E",
    ],

```

```

    ],
}

ACS_VARS = sorted({v for group in ACS_VAR_GROUPS.values() for v in group})
print("ACS variable count:", len(ACS_VARS))

```

ACS variable count: 98

Cell 5 — ACS API helpers

```

In [40]: def chunks(lst, n):
    for i in range(0, len(lst), n):
        yield lst[i : i + n]

def infer_state_fips_from_tracts(tracts):
    if not tracts:
        return None
    return sorted({str(t).zfill(11)[:2] for t in tracts})

def resolve_acs_years(igs_years, max_available_year=ACS_MAX_AVAILABLE_YEAR):
    igs_years = sorted({int(y) for y in igs_years if pd.notna(y)})
    acs_years = [y for y in igs_years if y <= max_available_year]
    return acs_years

def get_state_fips(year: int, api_key: str) -> list[str]:
    url = f"https://api.census.gov/data/{year}/acs/acs5"
    params = {"get": "NAME", "for": "state:*", "key": api_key}

    r = requests.get(url, params=params, timeout=60)
    r.raise_for_status()

    data = r.json()
    header, rows = data[0], data[1:]
    df = pd.DataFrame(rows, columns=header)

    return sorted(df["state"].unique().tolist())

def census_get_tracts(year: int, state_fips: str, vars_: list[str], api_key:
    base = f"https://api.census.gov/data/{year}/acs/acs5"
    get_str = "NAME," + ",".join(vars_)

    params = [
        ("get", get_str),
        ("for", "tract:*"),
        ("in", f"state:{state_fips}"),
        ("in", "county:*"),
        ("key", api_key),
    ]

    r = requests.get(base, params=params, timeout=120)
    r.raise_for_status()

```

```

data = r.json()
header, rows = data[0], data[1:]
df = pd.DataFrame(rows, columns=header)
df["year"] = year

return df

def to_numeric_safe(df: pd.DataFrame, exclude=("NAME", "state", "county", "t
out = df.copy()
for c in out.columns:
    if c in exclude:
        continue
    out[c] = pd.to_numeric(out[c], errors="coerce")
return out

ACS_SENTINELS = {-666666666, -333333333, -222222222}

def clean_acs_values(df: pd.DataFrame, exclude=("NAME", "state", "county", "
out = df.copy()
for c in out.columns:
    if c in exclude:
        continue
    out[c] = pd.to_numeric(out[c], errors="coerce")
    out[c] = out[c].replace(list(ACS_SENTINELS), np.nan)
return out

def download_acs_all(
    years: list[int],
    api_key: str,
    state_fips_filter: list[str] | None = None,
    max_vars_per_call: int = 45,
    sleep_seconds: float = 0.25,
) -> pd.DataFrame:
    if not api_key:
        raise ValueError("Missing CENSUS_API_KEY. Put it in your environment

    all_parts = []

    for year in years:
        year_out = RAW_DIR / f"acs5_tract_{year}.parquet"

        if year_out.exists():
            print(f"[cache] {year_out.name}")
            df_year = pd.read_parquet(year_out)
            all_parts.append(df_year)
            continue

        states = state_fips_filter if state_fips_filter else get_state_fips(
            print(f"[download] year={year} | states={len(states)}")

        state_frames = []

        for st in states:

```



```

merged_state = None

for var_chunk in chunks(ACS_VARS, max_vars_per_call):
    df_chunk = census_get_tracts(year, st, var_chunk, api_key)
    df_chunk = clean_acs_values(df_chunk)

    keys = ["state", "county", "tract", "NAME", "year"]
    if merged_state is None:
        merged_state = df_chunk
    else:
        merged_state = merged_state.merge(df_chunk, on=keys, how='outer')

    time.sleep(sleep_seconds)

state_frames.append(merged_state)

df_year = pd.concat(state_frames, ignore_index=True)

if SAVE_INTERMEDIATE:
    safe_save_parquet(df_year, year_out)

all_parts.append(df_year)

acs = pd.concat(all_parts, ignore_index=True)
return acs

```

Cell 6 — ACS feature engineering

```

In [41]: def safe_div(n, d):
    n = np.asarray(n, dtype="float64")
    d = np.asarray(d, dtype="float64")

    out = np.full_like(n, np.nan, dtype="float64")
    valid = (~np.isnan(d)) & (d != 0)
    np.divide(n, d, out=out, where=valid)
    return out

def make_features(acs: pd.DataFrame) -> pd.DataFrame:
    acs = acs.copy()

    acs["state"] = acs["state"].astype(str).str.zfill(2)
    acs["county"] = acs["county"].astype(str).str.zfill(3)
    acs["tract"] = acs["tract"].astype(str).str.zfill(6)
    acs["geoid"] = acs["state"] + acs["county"] + acs["tract"]

    out = pd.DataFrame({
        "geoid": acs["geoid"],
        "year": acs["year"],
    })

    # -----
    # Population / age
    # -----
    out["pop_total"] = acs["B01001_001E"]

```

```

out["pop_under18"] = (
    acs["B01001_003E"] + acs["B01001_004E"] + acs["B01001_005E"] + acs["
    acs["B01001_027E"] + acs["B01001_028E"] + acs["B01001_029E"] + acs["
)
out["share_under18"] = safe_div(out["pop_under18"], out["pop_total"])

out["pop_65plus"] = (
    acs["B01001_020E"] + acs["B01001_021E"] + acs["B01001_022E"] + acs["
    acs["B01001_044E"] + acs["B01001_045E"] + acs["B01001_046E"] + acs["
)
out["share_65plus"] = safe_div(out["pop_65plus"], out["pop_total"])

# -----
# Race shares
# -----
out["share_white"] = safe_div(acs["B02001_002E"], acs["B02001_001E"])
out["share_black"] = safe_div(acs["B02001_003E"], acs["B02001_001E"])
out["share_asian"] = safe_div(acs["B02001_005E"], acs["B02001_001E"])
out["share_two_plus"] = safe_div(acs["B02001_008E"], acs["B02001_001E"])

# -----
# Education
# -----
ba_plus = acs["B15003_022E"] + acs["B15003_023E"] + acs["B15003_024E"] +
out["ba_plus_share_25p"] = safe_div(ba_plus, acs["B15003_001E"])

# -----
# Employment / economy
# -----
out["lfpr_16p"] = safe_div(acs["B23025_002E"], acs["B23025_001E"])
out["unemp_rate"] = safe_div(acs["B23025_005E"], acs["B23025_003E"])

out["median_household_income"] = acs["B19013_001E"]
out["per_capita_income"] = acs["B19301_001E"]
out["gini"] = acs["B19083_001E"]
out["poverty_rate"] = safe_div(acs["B17001_002E"], acs["B17001_001E"])

# -----
# Commute / internet
# -----
commute_under35 = (
    acs["B08303_002E"] + acs["B08303_003E"] + acs["B08303_004E"] + acs["
    acs["B08303_006E"] + acs["B08303_007E"] + acs["B08303_008E"]
)
out["commute_under35_share"] = safe_div(commute_under35, acs["B08303_001
out["internet_sub_share"] = safe_div(acs["B28002_002E"], acs["B28002_001

# -----
# Housing / affordability
# -----
out["housing_units_total"] = acs["B25002_001E"]
out["occupied_units"] = acs["B25002_002E"]
out["vacant_units"] = acs["B25002_003E"]
out["occupied_share"] = safe_div(acs["B25002_002E"], acs["B25002_001E"])
out["vacancy_rate"] = safe_div(acs["B25002_003E"], acs["B25002_001E"])

```

```

out["owner_share"] = safe_div(acs["B25003_002E"], acs["B25003_001E"])
out["renter_share"] = safe_div(acs["B25003_003E"], acs["B25003_001E"])

out["median_gross_rent"] = acs["B25064_001E"]
out["median_home_value"] = acs["B25077_001E"]

rent_affordable = (
    acs["B25070_002E"] + acs["B25070_003E"] + acs["B25070_004E"] +
    acs["B25070_005E"] + acs["B25070_006E"]
)

owner_affordable = (
    acs["B25091_003E"] + acs["B25091_004E"] + acs["B25091_005E"] + acs["
    acs["B25091_014E"] + acs["B25091_015E"] + acs["B25091_016E"] + acs["
)

denom_housing_cost = acs["B25070_001E"] + acs["B25091_001E"]
out["affordable_housing_share"] = safe_div(rent_affordable + owner_affor

# -----
# Community / health
# -----
# Early education proxy: enrolled 3-4 year olds over under-5 population
enrolled_3_4 = acs["B14003_004E"] + acs["B14003_013E"] + acs["B14003_032
pop_under5 = acs["B01001_003E"] + acs["B01001_027E"]
out["early_ed_enroll_share"] = safe_div(enrolled_3_4, pop_under5)

uninsured = (
    acs["B27001_005E"] + acs["B27001_008E"] + acs["B27001_011E"] + acs["
    acs["B27001_020E"] + acs["B27001_023E"] + acs["B27001_026E"] + acs["
    acs["B27001_033E"] + acs["B27001_036E"] + acs["B27001_039E"] + acs["
    acs["B27001_048E"] + acs["B27001_051E"] + acs["B27001_054E"] + acs["
)
out["insured_share"] = 1.0 - safe_div(uninsured, acs["B27001_001E"])

# -----
# Growth features
# -----
out = out.sort_values(["geoid", "year"]).reset_index(drop=True)

growth_cols = [
    "median_household_income",
    "per_capita_income",
    "occupied_units",
    "median_home_value",
    "median_gross_rent",
]

for col in growth_cols:
    out[f"{col}_growth"] = out.groupby("geoid")[col].pct_change(fill_met

return out

```

Cell 7 — Merge helpers

```
In [42]: def prepare_acs_raw_for_merge(acs: pd.DataFrame) -> pd.DataFrame:
    acs_raw = acs.copy()

    acs_raw["state"] = acs_raw["state"].astype(str).str.zfill(2)
    acs_raw["county"] = acs_raw["county"].astype(str).str.zfill(3)
    acs_raw["tract"] = acs_raw["tract"].astype(str).str.zfill(6)
    acs_raw["geoid"] = acs_raw["state"] + acs_raw["county"] + acs_raw["tract"]

    # one row per tract-year expected
    acs_raw = acs_raw.drop_duplicates(subset=["geoid", "year"]).copy()

    return acs_raw

def merge_igs_acs_raw(igs: pd.DataFrame, acs_raw: pd.DataFrame) -> pd.DataFrame:
    merged = igs.merge(acs_raw, on=["geoid", "year"], how="left", validate="m:m")
    return merged

def merge_igs_acs_full(
    igs: pd.DataFrame,
    acs_raw: pd.DataFrame,
    feats: pd.DataFrame,
) -> pd.DataFrame:
    merged = (
        igs
        .merge(acs_raw, on=["geoid", "year"], how="left", validate="m:m")
        .merge(feats, on=["geoid", "year"], how="left", validate="m:m")
    )
    return merged
```

```
In [43]: # for year in range(2017, 2025):
#         p = RAW_DIR / f"acs5_tract_{year}.parquet"
#         if p.exists():
#             p.unlink()
#             print("deleted", p.name)
```

Cell 8 — Run ACS pull, feature engineering, and merge

```
In [44]: igs_years = sorted(igs["year"].dropna().unique().tolist())
acs_years = resolve_acs_years(igs_years, max_available_year=ACS_MAX_AVAILABLE_YEAR)
state_fips_filter = infer_state_fips_from_tracts(TARGET_TRACTS)

print("IGS years in file:", igs_years)
print("ACS years to pull:", acs_years)
print("State filter from target tracts:", state_fips_filter)

acs = download_acs_all(
    years=acs_years,
    api_key=CENSUS_API_KEY,
    state_fips_filter=state_fips_filter,
    max_vars_per_call=45,
    sleep_seconds=0.25,
```

```

)

acs_raw = prepare_acs_raw_for_merge(acs)
feats = make_features(acs)
feats = filter_tracts(feats, TARGET_TRACTS)

# keep only IGS years that exist in ACS-derived tables
valid_years = sorted(feats["year"].dropna().unique())
model_igs = igs[igs["year"].isin(valid_years)].copy()

igs_x_acs_raw = merge_igs_acs_raw(model_igs, acs_raw)
model_df_full = merge_igs_acs_full(model_igs, acs_raw, feats)

if SAVE_INTERMEDIATE:
    safe_save_parquet(acs, OUT_DIR / "acs_raw.parquet")
    safe_save_parquet(acs_raw, OUT_DIR / "acs_raw_with_geoid.parquet")
    safe_save_parquet(feats, OUT_DIR / "acs_features.parquet")
    safe_save_parquet(igs_x_acs_raw, OUT_DIR / "igs_x_acs_raw.parquet")
    safe_save_parquet(model_df_full, OUT_DIR / "igs_x_acs_full.parquet")

print("ACS raw shape:", acs.shape)
print("ACS raw+geoid shape:", acs_raw.shape)
print("ACS features shape:", feats.shape)
print("IGS x ACS raw shape:", igs_x_acs_raw.shape)
print("Full merged model_df_full shape:", model_df_full.shape)

```

```

IGS years in file: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]
ACS years to pull: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]
State filter from target tracts: None
[cache] acs5_tract_2017.parquet
[cache] acs5_tract_2018.parquet
[cache] acs5_tract_2019.parquet
[cache] acs5_tract_2020.parquet
[cache] acs5_tract_2021.parquet
[cache] acs5_tract_2022.parquet
[cache] acs5_tract_2023.parquet
[cache] acs5_tract_2024.parquet
ACS raw shape: (648952, 103)
ACS raw+geoid shape: (648952, 104)
ACS features shape: (648952, 37)
IGS x ACS raw shape: (680256, 189)
Full merged model_df_full shape: (680256, 224)

```

validation cell right after Cell 8

```

In [45]: # choose the final EDA panel
eda_df = model_df_full.copy()

print("EDA shape:", eda_df.shape)
print("EDA years:", sorted(eda_df["year"].dropna().unique().tolist()))
print("EDA unique tracts:", eda_df["geoid"].nunique())

# required ACS raw columns
required_acs_raw = ["NAME", "state", "county", "tract"] + ACS_VARS
missing_acs_raw = [c for c in required_acs_raw if c not in eda_df.columns]

```

```

# required engineered columns
required_feats = [c for c in feats.columns if c not in ["geoid", "year"]]
missing_feats = [c for c in required_feats if c not in eda_df.columns]

# all original IGS columns
missing_igs = [c for c in igs.columns if c not in eda_df.columns]

print("Missing raw ACS columns:", missing_acs_raw[:20], "count =", len(missing_acs_raw))
print("Missing engineered feature columns:", missing_feats[:20], "count =", len(missing_feats))
print("Missing IGS columns:", missing_igs[:20], "count =", len(missing_igs))

print("Duplicate geoid-year rows:", eda_df.duplicated(subset=["geoid", "year"]).sum())

# quick sample
cols_to_show = [
    "geoid", "year", "igs_total", "igs_place", "igs_economy", "igs_community",
    "NAME", "state", "county", "tract",
    "B19013_001E", "B19301_001E", "B25077_001E",
    "median_household_income", "per_capita_income", "median_home_value"
]
print(eda_df[cols_to_show].head())

```

EDA shape: (680256, 224)

EDA years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]

EDA unique tracts: 85032

Missing raw ACS columns: [] count = 0

Missing engineered feature columns: [] count = 0

Missing IGS columns: [] count = 0

Duplicate geoid-year rows: 0

	geoid	year	igs_total	igs_place	igs_economy	igs_community	
NAME	state	county	tract	B19013_001E	B19301_001E	B25077_001E	\
0	01001020100	2017	47.0	53.0	34.0	54.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	67826.0
33018.0			152500.0				
1	01001020100	2018	52.0	48.0	48.0	62.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	58625.0
31580.0			133300.0				
2	01001020100	2019	46.0	40.0	40.0	56.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	60208.0
31225.0			136100.0				
3	01001020100	2020	46.0	34.0	48.0	56.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	60388.0
30447.0			151200.0				
4	01001020100	2021	38.0	43.0	32.0	40.0	Cens
us Tract 201, Autauga County, Alabama				01	001	020100	57399.0
30934.0			138700.0				

	median_household_income	per_capita_income	median_home_value
0	67826.0	33018.0	152500.0
1	58625.0	31580.0	133300.0
2	60208.0	31225.0	136100.0
3	60388.0	30447.0	151200.0
4	57399.0	30934.0	138700.0

Cell 9 — Final checks before EDA

```
In [46]: print("Merged years:", sorted(model_df_full["year"].dropna().unique().tolist))
print()

print(model_df_full[[
    "geoid", "year", "igs_total", "igs_place", "igs_economy", "igs_community",
    "affordable_housing_share", "internet_sub_share", "insured_share",
    "median_household_income", "poverty_rate", "vacancy_rate"
]].head())

print()
print("Top missingness in merged data:")
print(model_df_full.isna().mean().sort_values(ascending=False).head(30))

print()
print("Core numeric summary:")
core_cols = [
    "igs_total",
    "igs_place",
    "igs_economy",
    "igs_community",
    "affordable_housing_share",
    "internet_sub_share",
    "insured_share",
    "median_household_income",
    "per_capita_income",
    "poverty_rate",
    "vacancy_rate",
    "unemp_rate",
]
print(model_df_full[core_cols].describe())
```

Merged years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]

	geoid	year	igs_total	igs_place	igs_economy	igs_community	affordable_housing_share	internet_sub_share	insured_share	median_household_income	poverty_rate	vacancy_rate
0	01001020100	2017	47.0	53.0	34.0	54.0	0.737401	0.762599	0.909485	67826.0	0.106775	0.014379
1	01001020100	2018	52.0	48.0	48.0	62.0	0.704575	0.717647	0.907436	58625.0	0.113365	0.017972
2	01001020100	2019	46.0	40.0	40.0	56.0	0.715092	0.740480	0.902659	60208.0	0.166583	0.022069
3	01001020100	2020	46.0	34.0	48.0	56.0	0.738817	0.810967	0.903658	60388.0	0.136528	0.023944
4	01001020100	2021	38.0	43.0	32.0	40.0	0.743842	0.840722	0.909548	57399.0	0.153546	0.057276

Top missingness in merged data:

Is an Opportunity Zone	1.000000
Spending per Capita Tract, %	1.000000
Spend Growth Base, %	1.000000
is_opp_zone	1.000000
Spending per Capita Base, %	1.000000
Spend Growth Tract, %	1.000000
median_gross_rent_growth	0.276211
median_home_value_growth	0.256606
median_household_income_growth	0.241214
occupied_units_growth	0.236046
per_capita_income_growth	0.234822
Minority/Women Owned Businesses Tract, %	0.159347
Minority/Women Owned Businesses Base, %	0.159347
Minority/Women Owned Businesses Score	0.159347
Early Education Enrollment Tract, %	0.157891
Early Education Enrollment Score	0.157891
B25064_001E	0.147337
median_gross_rent	0.147337
B25077_001E	0.131784
median_home_value	0.131784
early_ed_enroll_share	0.124228
B19013_001E	0.116411
median_household_income	0.116411
gini	0.113279
B19083_001E	0.113279
affordable_housing_share	0.112346
internet_sub_share	0.112336
owner_share	0.112336
renter_share	0.112336
occupied_share	0.112055
dtype: float64	

Core numeric summary:

igs_total	igs_place	igs_economy	igs_community	affordable_housing_share
-----------	-----------	-------------	---------------	--------------------------

e_housing_share	internet_sub_share	insured_share	median_household_income	per_capita_income	poverty_rate \
count	674227.000000	677454.000000	679363.000000	677817.000000	
	603832.000000	603839.000000	604557.000000	601067.000000	
	605097.000000	604202.000000			
mean	50.092197	49.971285	50.313671	50.150358	
	0.654425	0.847756	0.910665	73893.783232	3
	7332.819006	0.145232			
std	10.325957	12.498407	13.613307	13.857690	
	0.129589	0.115830	0.073407	37973.270136	2
	0408.309777	0.120057			
min	0.000000	0.000000	0.000000	0.000000	
	0.000000	0.000000	0.000000	2499.000000	
	0.000000	0.000000			
25%	44.000000	42.000000	42.000000	40.000000	
	0.575375	0.789683	0.879251	47967.000000	2
	4444.000000	0.059760			
50%	50.100000	50.000000	51.300000	50.000000	
	0.673483	0.873674	0.930211	65578.000000	3
	2765.000000	0.111340			
75%	58.000000	58.000000	60.000000	60.000000	
	0.750096	0.933931	0.962620	90962.000000	4
	4589.000000	0.195118			
max	84.000000	100.000000	94.000000	100.000000	
	1.000000	1.000000	1.000000	250001.000000	69
	1205.000000	1.000000			

	vacancy_rate	unemp_rate
count	604030.000000	604505.000000
mean	0.111957	0.060183
std	0.107473	0.049349
min	0.000000	0.000000
25%	0.042771	0.028459
50%	0.082713	0.048499
75%	0.145941	0.077549
max	1.000000	1.000000

Cell 10 — Optional: save an EDA subset for only chosen tracts

```
In [47]: # Cell 10 – final EDA panel cleanup + save

eda_df = model_df_full.copy() # use model_df_full, not model_df

# drop junk unnamed / NaN column names from the original IGS export
junk_cols = [c for c in eda_df.columns if pd.isna(c) or str(c).startswith("unnamed:")]
print("Dropping junk columns:", junk_cols)

eda_df = eda_df.drop(columns=junk_cols, errors="ignore")

# optional: reorder key columns to the front
front_cols = [
    c for c in [
        "geoid", "year", "Census Tract FIPS code", "County", "State",
        "igs_total", "igs_place", "igs_economy", "igs_community"
    ]
```

```

    ] if c in eda_df.columns
]
other_cols = [c for c in eda_df.columns if c not in front_cols]
eda_df = eda_df[front_cols + other_cols]

print("EDA panel shape:", eda_df.shape)

if SAVE_INTERMEDIATE:
    safe_save_parquet(eda_df, OUT_DIR / "eda_panel_clean.parquet")

eda_df.head()

```

Dropping junk columns: []
EDA panel shape: (680256, 224)

Out[47]:

	geoid	year	Census Tract FIPS code	County	State	igs_total	igs_place	igs
0	01001020100	2017	01001020100	Autauga County	Alabama	47.0	53.0	
1	01001020100	2018	01001020100	Autauga County	Alabama	52.0	48.0	
2	01001020100	2019	01001020100	Autauga County	Alabama	46.0	40.0	
3	01001020100	2020	01001020100	Autauga County	Alabama	46.0	34.0	
4	01001020100	2021	01001020100	Autauga County	Alabama	38.0	43.0	

5 rows × 224 columns

Cell 11 — validation checks

In [48]: *# Cell 11 – validation checks*

```
print("Duplicate geoid-year rows:", eda_df.duplicated(subset=["geoid", "year"]  
  
required_acs_raw = ["NAME", "state", "county", "tract"] + ACS_VARS  
missing_acs_raw = [c for c in required_acs_raw if c not in eda_df.columns]  
  
required_feats = [c for c in feats.columns if c not in ["geoid", "year"]]  
missing_feats = [c for c in required_feats if c not in eda_df.columns]  
  
missing_igs = [c for c in igs.columns if c not in eda_df.columns]  
  
print("Missing raw ACS columns:", len(missing_acs_raw))  
print("Missing engineered ACS columns:", len(missing_feats))  
print("Missing IGS columns:", len(missing_igs))
```

Duplicate geoid-year rows: 0
Missing raw ACS columns: 0
Missing engineered ACS columns: 0
Missing IGS columns: 0

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: